

Categorical Semantics and Operational Soundness for an Extended Simply Typed Lambda Calculus in Lean 4

Soma Saito

Abstract

We formalize categorical semantics for intrinsically typed terms and typed substitutions of `STLCextBool` in a chosen Cartesian closed category equipped with primitive Boolean structure. The semantic carrier is the intrinsically typed syntax `ITerm.Core`, whose indices retain typing information. Consequently, variable lookup, weakening, lifting under binders, and simultaneous substitution can be defined without eliminating typing proofs into computational data.

Our main results are semantic substitution, denotational soundness of lambda beta reduction and eta expansion, and soundness of the two primitive Boolean branching rules. We further define `CoreStep`, the full congruence closure of beta and Boolean root reduction, together with a call-by-value relation `CoreCBVStep`. One-step and finite multi-step executions in both relations preserve denotation. Erasure maps the intrinsic relations to the existing `Step` and `CBVStep` relations. In the reverse direction, every existing `Step` from a fixed intrinsic source admits an intrinsically typed lift. The development is machine-checked in Lean 4.24.0 and built as part of the Metatheory library.

1 Introduction

The correspondence between operational and categorical semantics for typed lambda calculi is a concrete manifestation of the Curry–Howard–Lambek correspondence. In the simply typed lambda calculus, types are interpreted as objects, terms as morphisms, application by evaluation, and abstraction by currying in a Cartesian closed category (CCC) [19, 4, 18].

A machine-checked account in Lean 4 must, however, coordinate binders, de Bruijn indices, simultaneous substitutions, context extension, and operational reduction. A direct interpretation of an untyped raw term together with a proposition-valued typing derivation encounters a design obstacle: the denotation returns data in a type determined by the derivation, while Lean restricts elimination from propositions into such computational data. More generally, representations of binding and the associated substitution lemmas are central engineering choices in mechanized metatheory [5, 3].

We place the intrinsically typed syntax of `STLCextBool` at the center of the semantics. Its relation to the pre-existing untyped syntax is factored into erasure and typed lifting of operational steps. This factorization lets us state semantic results directly over typed terms while reusing the existing `Step` and `CBVStep` APIs.

The contributions are:

1. a denotation of intrinsically typed `STLCextBool` terms in any chosen CCC with primitive Boolean structure;
2. typed substitutions and semantic accounts of lookup, weakening, and lifting under binders;
3. machine-checked semantic substitution and soundness of lambda beta, lambda eta, and primitive Boolean computation;

4. denotational preservation for full-congruence and call-by-value one-step and multi-step relations, connected by erasure and typed lifting to the library's existing operational semantics.

We do not prove initiality of a syntactic category, completeness of the semantics, computational adequacy, or a coproduct characterization of Booleans. The contribution is the verified bridge among intrinsic syntax, categorical substitution, and existing operational relations.

2 Preliminaries

2.1 Types, contexts, and intrinsically typed terms

Types consist of base types, function types, and a primitive Boolean type:

$$A, B ::= \text{base}(n) \mid A \Rightarrow B \mid \text{Bool} \quad (n \in \mathbb{N}).$$

Contexts are lists of types. We use a head-first convention, so the extended context $A :: \Gamma$ is interpreted as the product of the interpretation of A with that of Γ .

For each model, fix an interpretation $\text{base}_C : \mathbb{N} \rightarrow \text{Obj}(C)$ of base types and define

$$\llbracket \text{base}(n) \rrbracket_C = \text{base}_C(n), \quad \llbracket A \Rightarrow B \rrbracket_C = \llbracket B \rrbracket_C^{[A]_C}, \quad \llbracket \text{Bool} \rrbracket_C = \text{Bool}_C.$$

The corresponding Lean definitions are `STLCextBool.Ty` and `BoolCCCModel.interpretTy`.

An intrinsically typed term has type `ITerm  $\Gamma$  A`. Variables carry evidence of typed lookup in a context; application records the function and argument types; abstraction stores a body in an extended context; and the Boolean extension provides `true`, `false`, and `if`.

The principal formation rules are

$$\begin{array}{c} \dfrac{i : \text{Var } \Gamma \ A}{\Gamma \vdash_{\text{int}} i : A} \quad \dfrac{\Gamma \vdash_{\text{int}} f : A \Rightarrow B \quad \Gamma \vdash_{\text{int}} a : A}{\Gamma \vdash_{\text{int}} f a : B}, \\[10pt] \dfrac{A :: \Gamma \vdash_{\text{int}} b : B}{\Gamma \vdash_{\text{int}} \lambda b : A \Rightarrow B} \quad \dfrac{}{\Gamma \vdash_{\text{int}} \text{true} : \text{Bool}} \quad \dfrac{}{\Gamma \vdash_{\text{int}} \text{false} : \text{Bool}}, \\[10pt] \dfrac{\Gamma \vdash_{\text{int}} c : \text{Bool} \quad \Gamma \vdash_{\text{int}} t : A \quad \Gamma \vdash_{\text{int}} e : A}{\Gamma \vdash_{\text{int}} \text{if } c \text{ then } t \text{ else } e : A}. \end{array}$$

Variables use a de Bruijn-style representation. This approach goes back to de Bruijn and has been refined into typed, scope-safe representations [9, 7, 3]. Ill-typed applications and conditionals with differently typed branches are not elements of `ITerm.Core`; Lean rejects their construction.

This choice does not discard the existing raw syntax `STLCextBool.Term`. Erasure maps `ITerm.Core` to the raw syntax and is proved compatible with typing. We define denotation and typed operational relations intrinsically, then connect them to the existing relations by separate erasure and lifting theorems.

2.2 Chosen Cartesian closed structure

A chosen CCC model provides objects, morphisms, identities, composition, a terminal object, binary products, exponentials, evaluation, currying and uncurrying, and their beta and eta laws. “Chosen” means that data ordinarily determined only up to isomorphism is fixed as concrete structure that Lean can refer to directly [19, 4].

We write

$$\llbracket A \rrbracket_C, \quad \llbracket \Gamma \rrbracket_C$$

for interpretations of types and contexts. The empty context is sent to the terminal object, and

$$\llbracket A :: \Gamma \rrbracket_C = \llbracket A \rrbracket_C \times \llbracket \Gamma \rrbracket_C.$$

Throughout the paper, $f; g$ means “first f , then g ”, namely $g \circ f$ in conventional notation. The categorical laws used below are

$$\begin{aligned} \text{id}_X; f &= f, & f; \text{id}_Y &= f, & (f; g); h &= f; (g; h), \\ \langle f, g \rangle; \pi_1 &= f, & \langle f, g \rangle; \pi_2 &= g, & \langle r; \pi_1, r; \pi_2 \rangle &= r, \end{aligned}$$

and

$$\text{uncurry}(\text{curry}(q)) = q, \quad \text{curry}(\text{uncurry}(r)) = r.$$

We also use

$$\text{uncurry}(r) = \langle \pi_1; r, \pi_2 \rangle; \text{eval}.$$

Because context extension is ordered as $\llbracket A \rrbracket_C \times \llbracket \Gamma \rrbracket_C$, while currying consumes a morphism whose domain is ordered as $\llbracket \Gamma \rrbracket_C \times \llbracket A \rrbracket_C$, abstraction uses an explicit exchange morphism $\text{swap}_{A, \Gamma}$. Keeping this morphism visible is essential in the semantic substitution proof.

2.3 Primitive Boolean structure

Primitive Boolean structure consists of:

- an object Bool_C ;
- two points $\text{truePoint}, \text{falsePoint} : 1 \rightarrow \text{Bool}_C$;
- an if-eliminator selecting between two morphisms with a common domain and codomain;
- naturality of that eliminator under precomposition;
- computation rules for the two Boolean points.

For $c : X \rightarrow \text{Bool}_C$ and $t, e : X \rightarrow A$, write

$$\text{ite}_{X,A}(c, t, e) : X \rightarrow A.$$

The computation laws are

$$\text{ite}_{X,A}(!_X; \text{truePoint}, t, e) = t, \tag{1}$$

$$\text{ite}_{X,A}(!_X; \text{falsePoint}, t, e) = e. \tag{2}$$

For $h : Y \rightarrow X$, naturality requires

$$h; \text{ite}_{X,A}(c, t, e) = \text{ite}_{Y,A}(h; c, h; t, h; e). \tag{3}$$

Equation (3) is exactly the categorical law used in the `if` case of semantic substitution.

In Lean, `BoolCCModel` stores this data in the fields `boolObj`, `truePoint`, `falsePoint`, `iteOp`, `ite_true`, `ite_false`, and `ite_natural`.

Remark 2.1. This interface does not construct Booleans as a binary coproduct and does not claim the universal property $\text{Bool} \cong 1 + 1$. The weaker chosen interface is sufficient for interpreting `true`, `false`, and `if` and for proving the computation rules used in this paper.

3 Formalization

3.1 Denotation of terms

The denotation of a term is a morphism from the context object to the object interpreting its type:

$$\llbracket t \rrbracket_C : \llbracket \Gamma \rrbracket_C \longrightarrow \llbracket A \rrbracket_C \quad (t : \text{ITerm } \Gamma \ A).$$

Writing $X_\Gamma = \llbracket \Gamma \rrbracket_C$, denotation is defined by the equations

$$\begin{aligned} \llbracket \text{var}(\text{zero}) \rrbracket_C &= \pi_1, \\ \llbracket \text{var}(\text{succ}(v)) \rrbracket_C &= \pi_2; \llbracket \text{var}(v) \rrbracket_C, \\ \llbracket \text{app}(f, a) \rrbracket_C &= \langle \llbracket f \rrbracket_C, \llbracket a \rrbracket_C \rangle; \text{eval}, \\ \llbracket \text{lam}(b) \rrbracket_C &= \text{curry}(\text{swap}_{A, \Gamma}; \llbracket b \rrbracket_C), \\ \llbracket \text{true} \rrbracket_C &= !_X; \text{truePoint}, \\ \llbracket \text{false} \rrbracket_C &= !_X; \text{falsePoint}, \\ \llbracket \text{if}(c, t, e) \rrbracket_C &= \text{ite}_{X_\Gamma, \llbracket A \rrbracket_C}(\llbracket c \rrbracket_C, \llbracket t \rrbracket_C, \llbracket e \rrbracket_C). \end{aligned}$$

The first and second projections represent the newest variable and the remaining context, respectively. The explicit swap in the abstraction case reconciles the head-first context convention with the argument order required by currying.

Remark 3.1 (Typing of denotation). For $t : \text{ITerm.Core } \Gamma \ A$, the expression $\llbracket t \rrbracket_C$ has type $\text{Hom}_C(\llbracket \Gamma \rrbracket_C, \llbracket A \rrbracket_C)$. This is not a separate post hoc theorem: it is the return type of the Lean definition `ITerm.denote`.

3.2 Typed substitutions

A typed substitution $\sigma : \text{ISubst } \Gamma \ \Delta$ maps every variable in Δ to a term over Γ . Under a binder, $\sigma.\text{up } A$ preserves the new head variable and weakens every old component.

The denotation

$$\llbracket \sigma \rrbracket_C : \llbracket \Gamma \rrbracket_C \longrightarrow \llbracket \Delta \rrbracket_C$$

is defined by

$$\begin{aligned} \llbracket \text{nil} \rrbracket_C &= !_X; X_\Gamma \rightarrow 1, \\ \llbracket \text{cons}(t, \sigma) \rrbracket_C &= \langle \llbracket t \rrbracket_C, \llbracket \sigma \rrbracket_C \rangle. \end{aligned}$$

For a typed variable $v : \text{Var } \Delta \ A$, lookup satisfies

$$\llbracket \sigma.\text{lookup}(v) \rrbracket_C = \llbracket \sigma \rrbracket_C; \llbracket \text{var}(v) \rrbracket_C.$$

Lemma 3.2 (Weakening). For $t : \text{ITerm.Core } \Gamma \ B$,

$$\llbracket \text{weaken}_A(t) \rrbracket_C = \pi_2; \llbracket t \rrbracket_C.$$

The corresponding Lean theorem is `ITerm.denote_weaken_bool`.

Lemma 3.3 (Lifting a substitution). For $\sigma : \text{ISubst } \Gamma \ \Delta$,

$$\llbracket \sigma^\uparrow \rrbracket_C = \langle \pi_1, \pi_2; \llbracket \sigma \rrbracket_C \rangle : \llbracket A \rrbracket_C \times \llbracket \Gamma \rrbracket_C \longrightarrow \llbracket A \rrbracket_C \times \llbracket \Delta \rrbracket_C.$$

The corresponding Lean theorem is `denoteISubst_up_bool`. The formula separates two recurring actions: weakening ignores the new variable by precomposing with the second projection, whereas lifting retains that variable with the first projection and evaluates the old substitution after the second projection.

For example, if $\Delta = [A \Rightarrow B, A]$ and

$$\sigma = \text{cons}(f, \text{cons}(a, \text{nil})) : \text{ISubst } \Gamma \Delta,$$

then

$$\text{lookup}(\text{zero}, \sigma) = f, \quad \text{lookup}(\text{succ}(\text{zero}), \sigma) = a.$$

Under abstraction,

$$(\lambda b)[\sigma] = \lambda(b[\text{up}(A, \sigma)]),$$

so the newly bound variable is never replaced.

4 Main Results

The results have two layers. First, substitution, lambda beta and eta, and the primitive Boolean computation rules preserve denotation for intrinsically typed terms. Second, we define a full congruence closure and a call-by-value relation and prove one-step and finite multi-step preservation. Eta is an equational semantic principle, not an operational root reduction.

Theorem 4.1 (Semantic substitution). *For $t : \text{ITerm.Core } \Delta \ R$ and $\sigma : \text{ISubst } \Gamma \Delta$,*

$$\llbracket t[\sigma] \rrbracket_C = \llbracket \sigma \rrbracket_C ; \llbracket t \rrbracket_C.$$

The proof is by structural induction on t . The variable case uses the lookup theorem; application uses pairing and composition; and the `if` case uses the naturality law (3). The abstraction case is the central one. Put $S = \llbracket \sigma \rrbracket_C : X_\Gamma \rightarrow X_\Delta$ and $a = \llbracket A \rrbracket_C$. Lemma 3.3 yields

$$\llbracket [\sigma^\uparrow] \rrbracket_C = \langle \pi_1, \pi_2; S \rangle : a \times X_\Gamma \rightarrow a \times X_\Delta.$$

Currying expects the input order $X_\Gamma \times a$, so we explicitly use

$$\text{swap}_\Gamma : X_\Gamma \times a \rightarrow a \times X_\Gamma, \quad \text{swap}_\Delta : X_\Delta \times a \rightarrow a \times X_\Delta.$$

Naturality of exchange gives

$$\text{swap}_\Gamma ; \llbracket [\sigma^\uparrow] \rrbracket_C = \langle \pi_1; S, \pi_2 \rangle ; \text{swap}_\Delta,$$

and the precomposition law for currying gives

$$\text{curry}(\langle \pi_1; S, \pi_2 \rangle ; \text{swap}_\Delta ; \llbracket b \rrbracket_C) = S ; \text{curry}(\text{swap}_\Delta ; \llbracket b \rrbracket_C).$$

This is the required abstraction case. The Lean theorem is `denote_subst_bool`.

Theorem 4.2 (Soundness of lambda beta). *For $b : \text{ITerm.Core } (A :: \Gamma) \ B$ and $a : \text{ITerm.Core } \Gamma \ A$,*

$$\llbracket (\lambda b) a \rrbracket_C = \llbracket b[a] \rrbracket_C.$$

The beta substitution maps the head variable to a and all remaining variables identically. Its denotation is

$$\llbracket \text{betaSubst}(a) \rrbracket_C = \langle \llbracket a \rrbracket_C, \text{id}_{X_\Gamma} \rangle.$$

The theorem follows by combining semantic substitution with the beta law for evaluation and currying. Its Lean name is `intrinsic_beta_sound`.

Theorem 4.3 (Soundness of lambda eta). *For $f : \text{ITerm.Core } \Gamma \ (A \Rightarrow B)$,*

$$\llbracket \text{etaExpand}_A(f) \rrbracket_C = \llbracket f \rrbracket_C,$$

where

$$\text{etaExpand}_A(f) := \lambda(\text{app}(\text{weaken}_A(f), \text{var}(\text{zero}))).$$

The proof computes the pair formed by the weakened function and the new variable, simplifies the explicit swap, and applies $\text{curry}(\text{uncurry}(g)) = g$. Freshness of the eta variable is encoded in the intrinsic type. The Lean theorems are `intrinsic_eta_sound` and `coreEtaStep_denote`.

Theorem 4.4 (Soundness of primitive Boolean computation). *For any branches t, e ,*

$$\llbracket \text{if true then } t \text{ else } e \rrbracket_C = \llbracket t \rrbracket_C,$$

$$\llbracket \text{if false then } t \text{ else } e \rrbracket_C = \llbracket e \rrbracket_C.$$

These results apply the two chosen computation laws directly. Their Lean names are `intrinsic_ite_true_sound` and `intrinsic_ite_false_sound`.

4.1 Intrinsic operational relations

Write $t \rightarrow_r u$, $t \rightarrow_c u$, and $t \rightarrow_v u$ for `CoreRootStep`, `CoreStep`, and `CoreCBVStep`. Root reduction has the three rules

$$\frac{}{\text{app}(\text{lam}(b), a) \rightarrow_r b[\text{betaSubst}(a)]} \quad \frac{}{\text{if}(\text{true}, t, e) \rightarrow_r t} \quad \frac{}{\text{if}(\text{false}, t, e) \rightarrow_r e}.$$

`CoreStep` is the full congruence closure:

$$\begin{array}{c} \frac{t \rightarrow_r u}{t \rightarrow_c u} \quad \frac{f \rightarrow_c f'}{\text{app}(f, a) \rightarrow_c \text{app}(f', a)} \quad \frac{a \rightarrow_c a'}{\text{app}(f, a) \rightarrow_c \text{app}(f, a')}, \\ \frac{b \rightarrow_c b'}{\text{lam}(b) \rightarrow_c \text{lam}(b')} \quad \frac{c \rightarrow_c c'}{\text{if}(c, t, e) \rightarrow_c \text{if}(c', t, e)}, \\ \frac{t \rightarrow_c t'}{\text{if}(c, t, e) \rightarrow_c \text{if}(c, t', e)} \quad \frac{e \rightarrow_c e'}{\text{if}(c, t, e) \rightarrow_c \text{if}(c, t, e')}. \end{array}$$

Thus $\rightarrow_c$ is not an evaluation strategy: it is closed under abstraction and under both unselected branches of a conditional.

Values are abstractions and the two Boolean constants. The call-by-value relation is generated by

$$\begin{array}{c} \frac{\text{Val}(a)}{\text{app}(\text{lam}(b), a) \rightarrow_v b[\text{betaSubst}(a)]} \quad \frac{f \rightarrow_v f'}{\text{app}(f, a) \rightarrow_v \text{app}(f', a)}, \\ \frac{\text{Val}(f) \quad a \rightarrow_v a'}{\text{app}(f, a) \rightarrow_v \text{app}(f, a')} \quad \frac{c \rightarrow_v c'}{\text{if}(c, t, e) \rightarrow_v \text{if}(c', t, e)}, \\ \frac{}{\text{if}(\text{true}, t, e) \rightarrow_v t} \quad \frac{}{\text{if}(\text{false}, t, e) \rightarrow_v e}. \end{array}$$

By construction, $\rightarrow_v \subseteq \rightarrow_c$. Eta is included in neither relation.

Theorem 4.5 (CoreStep preserves denotation). *If CoreStep $t\ u$, then*

$$\llbracket t \rrbracket_C = \llbracket u \rrbracket_C.$$

The proof is by induction on the CoreStep derivation. Root cases use Theorem 4.2 and the two Boolean computation theorems. Each congruence case uses compositionality of denotation. For example, a step in the function position is transported through

$$\llbracket f\ a \rrbracket_C = \langle \llbracket f \rrbracket_C, \llbracket a \rrbracket_C \rangle; \text{eval}.$$

The Lean theorem is `coreStep_denote`.

Lemma 4.6 (Inclusion of call-by-value steps). *If CoreCBVStep $t\ u$, then CoreStep $t\ u$.*

The Lean theorem is `coreCBVStep_to_coreStep`.

Theorem 4.7 (CoreCBVStep preserves denotation). *If CoreCBVStep $t\ u$, then $\llbracket t \rrbracket_C = \llbracket u \rrbracket_C$.*

This follows from the inclusion lemma and Theorem 4.5; no separate call-by-value denotational semantics is required.

Theorem 4.8 (Finite multi-step preservation). *If Rewriting.Star CoreStep $t\ u$, then $\llbracket t \rrbracket_C = \llbracket u \rrbracket_C$. If Rewriting.Star CoreCBVStep $t\ u$, the same equality holds.*

Both claims follow by induction on the reflexive transitive closure and transitivity of equality. The Lean names are `core_step_star_denote` and `core_cbv_star_denote`.

Theorem 4.9 (Erasure and typed lifting). *CoreStep erases to the existing raw Step, and CoreCBVStep erases to the existing raw CBVStep. Moreover, for a fixed intrinsic source t and any raw step Step $t.\text{erase}\ N$, there exists an intrinsically typed target that erases to N together with a CoreStep from t to that target.*

The corresponding Lean theorems are `coreStep_erase`, `coreCBVStep_erase`, and `inverse_step_lift`. We do not claim the analogous reverse lifting theorem for arbitrary raw CBVStep. Nor do we claim that every raw typing derivation uniquely determines an intrinsic term. In Curry-style raw syntax, the intermediate type of an application cannot in general be recovered from the erased term alone.

5 Machine Verification and Reproducibility

The public Lean API is

`Metatheory/Paper2/Bridge.lean`.

Proof-bearing modules are isolated under `Metatheory/Paper2/Internal/`. The public bridge is imported from `Metatheory.lean`, so it is checked by the library build.

```
cd projects/paper1-metatheory
lake build Metatheory
./scripts/check.ps1
```

With Lean 4.24.0, `lake build Metatheory` completes 177 build jobs. The project check also reports no `sorry`, `admit`, or project-defined `axiom/constant` declarations in the Lean source. Lean 4 itself is described by de Moura and Ullrich [10].

The proof is divided into syntax, substitution, denotation, and operational layers. The proposition-valued raw typing relation is used to establish compatibility of erasure, not

Paper claim	Lean definition or theorem
Term denotation	<code>ITerm.denote</code>
Typing preservation of erasure	<code>ITerm.Core.erase_hasType</code>
Weakening and lifting	<code>Bridge.weakening_denotation,</code> <code>Bridge.lifted_substitution_denotation</code>
Semantic substitution	<code>Bridge.semantic_substitution</code>
Lambda beta	<code>Bridge.intrinsic_beta_denotation</code>
Lambda eta	<code>Bridge.intrinsic_eta_denotation</code>
Boolean branch computation	<code>intrinsic_ite_true_sound, intrinsic_ite_false_sound</code>
<code>CoreStep</code> preservation	<code>Bridge.intrinsic_step_denotation</code>
<code>CoreCBVStep</code> preservation	<code>Bridge.intrinsic_cbv_step_denotation</code>
Multi-step preservation	<code>Bridge.intrinsic_step_star_denotation,</code> <code>Bridge.intrinsic_cbv_step_star_denotation</code>
Erasure to existing relations	<code>Bridge.core_step_erase, Bridge.core_cbv_step_erase</code>
Lift from a fixed raw source	<code>Bridge.raw_step_intrinsic_lift_exists</code>

Table 1: Correspondence between paper claims and Lean declarations

as computational input to denotation. This separation makes the structural induction for substitution and denotation directly visible to Lean’s type checker.

All bridge names in Table 1 live in the namespace `Metatheory.Paper2.Bridge`. They re-export verified internal theorems without duplicating their proofs. The table does not imply uniqueness of a denotation for arbitrary raw terms or reverse lifting of every raw call-by-value step.

6 Related Work

6.1 Categorical semantics and syntax with binding

The interpretation of simply typed lambda calculus in CCCs is standard: contexts become products, function types become exponentials, abstraction becomes currying, and application becomes evaluation [4, 19, 18, 20]. Seely extends categorical semantics to higher-order polymorphism [22]; our language is the monomorphic `STLCextBool`. We do not claim novelty for the CCC interpretation itself or completeness of a syntactic category.

The formalization instead focuses on its interaction with de Bruijn-style intrinsic terms, typed substitutions, explicit exchange, and an existing operational API. De Bruijn notation absorbs alpha-conversion into the representation [9]; nested and scope-safe representations further encode scoping invariants in types [7, 3].

Fiore, Plotkin, and Turi characterize syntax with binding as an initial model with substitution structure [12]. Fiore and Szamozvancev develop this direction for second-order abstract syntax and an Agda formalization [13]. Our semantic substitution theorem is proved directly by structural induction on `ITerm.Core`; we do not derive it from an initiality theorem.

6.2 Operational semantics and mechanized categorical models

The call-by-name/call-by-value distinction goes back to Plotkin [21], while Wright and Felleisen systematize syntactic type soundness through progress and preservation [23]. Our objective is different: we prove that intrinsic reductions preserve categorical denotation and connect those reductions to existing raw relations. Operational type safety and categorical soundness are complementary but distinct results.

Ahrens and Zsidó formalize initial semantics for typed higher-order syntax in Coq [2]. Buisse and Dybjer study categorical models of dependent type theory in Agda [8], and

Ahrens, Lumsdaine, and Voevodsky compare categorical structures for type theory in Coq/UniMath [1]. Ekici mechanizes categorical reasoning for intuitionistic propositional logic in Cartesian closed preorders [11]. These developments emphasize initiality, dependent models, or logical reasoning rather than preservation for a concrete one-step relation of STLC.

Benton, Kennedy, and Varming verify operational and denotational results for a recursive call-by-value language in a domain-theoretic Coq model [6]. Gadea, Gunther, and Pagano connect intrinsic and extrinsic presentations and adequacy for a call-by-value language with Booleans [15]. Forster et al. combine weak and strong reduction, equational theory, and denotational semantics for CBPV in Coq [14]. These projects are close in their operational–denotational connection, but use concrete domain models or CBPV-specific structures rather than an arbitrary chosen CCC with the primitive Boolean interface used here.

In Lean 4, Ghalayini and Krishnaswami verify intrinsically typed syntax, typed substitution, denotational semantics, and semantic substitution for explicit refinement types [16]. Their work on SSA likewise connects typed syntax, substitution, equational theory, and categorical models [17]. Our development instead targets STLCextBool, is parameterized by a chosen CCC, and connects the semantic account to full-congruence and call-by-value operational relations already present in the Metatheory library.

To the best of our knowledge, the precise combination of primitive Boolean structure, intrinsically typed substitution, full-congruence and call-by-value preservation, erasure, and reverse lifting from a fixed raw source has not previously been machine-checked as one Lean 4 development. This is a bounded comparison, not an exhaustive priority claim.

7 Discussion and Limitations

The semantic carrier does not consume a raw typing derivation as computational data. Instead, the intrinsic term retains its type and context, while erasure and typed lifting provide the boundary to the existing raw syntax. This makes denotation and semantic substitution structurally recursive and removes repeated typing side conditions from application, abstraction, and conditionals.

The cost is concentrated at the operational boundary. Reconstructing an intrinsic target after a raw step requires rebuilding syntax and typing information simultaneously. We prove this reconstruction for an arbitrary raw `Step` from a fixed intrinsic source. We do not prove it for every raw `CBVStep`, and we do not prove witness independence for arbitrary raw typing derivations.

`CoreStep` includes beta and Boolean root computation and all application, abstraction, and conditional congruence rules. Therefore, Theorem 4.5 covers reduction at every syntactic position. `CoreCBVStep` is embedded into `CoreStep`, which yields denotational preservation without a second semantic interpretation. This does not establish operational adequacy, normalization, or completeness of the evaluation strategy.

Further work includes a stronger Boolean semantics characterized by finite coproducts, a complete reverse lifting for the existing call-by-value relation, and extensions to recursion or computational effects. Initiality of a free model, completeness, and integration with a general category-theory library remain outside the scope of this paper.

8 Conclusion

We gave categorical semantics for intrinsically typed STLCextBool terms and typed substitutions in a chosen CCC with primitive Boolean structure. Lean 4 machine-checks semantic substitution, lambda beta and eta soundness, primitive Boolean computation, and

denotational preservation for one-step and finite multi-step executions of both full-congruence and call-by-value relations. Erasure maps the intrinsic relations to the pre-existing raw **Step** and **CBVStep** APIs, and every raw **Step** from a fixed intrinsic source admits a typed reverse lift. The result is a reproducible operational–denotational bridge whose assumptions and limitations are explicit.

References

- [1] Benedikt Ahrens, Peter LeFanu Lumsdaine, and Vladimir Voevodsky. Categorical structures for type theory in univalent foundations. *Logical Methods in Computer Science*, 14(3:18):1–18, 2018.
- [2] Benedikt Ahrens and Julianna Zsidó. Initial semantics for higher-order typed syntax in Coq. *Journal of Formalized Reasoning*, 4(1):25–69, 2011.
- [3] Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. A type- and scope-safe universe of syntaxes with binding: Their semantics and proofs. *Journal of Functional Programming*, 31:e22, 2021.
- [4] Steve Awodey. *Category Theory*. Oxford University Press, 2010.
- [5] Brian Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. Engineering formal metatheory. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’08, pages 3–15. ACM, 2008.
- [6] Nick Benton, Andrew Kennedy, and Carsten Varming. Some domain theory and denotational semantics in Coq. In *Theorem Proving in Higher Order Logics*, volume 5674 of *Lecture Notes in Computer Science*, pages 115–130. Springer, 2009.
- [7] Richard S. Bird and Ross Paterson. de Bruijn notation as a nested datatype. *Journal of Functional Programming*, 9(1):77–91, 1999.
- [8] Alexandre Buisse and Peter Dybjer. Towards formalizing categorical models of type theory in type theory. *Electronic Notes in Theoretical Computer Science*, 196:137–151, 2008.
- [9] N. G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. *Indagationes Mathematicae (Proceedings)*, 75(5):381–392, 1972.
- [10] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [11] Burak Ekici. Formal categorical reasoning. *Turkish Journal of Mathematics*, 46(4):1538–1552, 2022.
- [12] Marcelo Fiore, Gordon Plotkin, and Daniele Turi. Abstract syntax and variable binding. In *Proceedings of the 14th Annual IEEE Symposium on Logic in Computer Science*, pages 193–202. IEEE Computer Society, 1999.
- [13] Marcelo Fiore and Dmitrij Szamozvancev. Formal metatheory of second-order abstract syntax. *Proceedings of the ACM on Programming Languages*, 6(POPL):1–29, 2022.
- [14] Yannick Forster, Steven Schäfer, Simon Spies, and Kathrin Stark. Call-by-push-value in Coq: Operational, equational, and denotational theory. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 118–131. Association for Computing Machinery, 2019.
- [15] Alejandro Gadea, Emmanuel Gunther, and Miguel Pagano. Mechanization of coherence and adequacy: Being extrinsic extended to subtyping. *Science of Computer Programming*, 197:102512, 2020.
- [16] Jad Elkhaleq Ghalayini and Neel Krishnaswami. Explicit refinement types. *Proceedings of the ACM on Programming Languages*, 7(ICFP):167–194, 2023.

- [17] Jad Elkhaleq Ghalayini and Neel Krishnaswami. The denotational semantics of SSA, 2024.
- [18] Bart Jacobs. *Categorical Logic and Type Theory*, volume 141 of *Studies in Logic and the Foundations of Mathematics*. Elsevier, Amsterdam, 1999.
- [19] Joachim Lambek and Philip J. Scott. *Introduction to Higher Order Categorical Logic*, volume 7 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, 1986.
- [20] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [21] Gordon D. Plotkin. Call-by-name, call-by-value and the lambda-calculus. *Theoretical Computer Science*, 1(2):125–159, 1975.
- [22] R. A. G. Seely. Categorical semantics for higher order polymorphic lambda calculus. *The Journal of Symbolic Logic*, 52(4):969–989, 1987.
- [23] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.